Cold Chain Intelligence

Product Requirements | Client: Contextual IO, for Harbor Table Foods | Team: SEC02_Contextual_Odyssey

1 Problem Statement

Harbor Table Foods makes ready-to-eat and prepared foods at three plants, each using ammonia refrigeration to chill its coolers and freezers. Ready-to-eat coolers hold 35 to 36°F; product above 41°F is outside safe cold holding. Ammonia leaks endanger staff: one reached 272 ppm, and 300 ppm is immediately dangerous. Controllers export text reports each night; a person reads them and writes the daily review next morning. Sudden faults are found only then, such as a receiver that read 0% for five hours during production while every cooler warmed. Slow faults never trip an alarm, such as a 02:45 freezer spike that grew from about 24°F to 49°F over ten days. Broken sensors also send false readings that look like emergencies. Many unusual readings can only be explained by checking other records, such as maintenance work, production schedules and weather.

2 Objective

Build a system on the Contextual platform that reads the nightly data, finds real faults early, separates them from sensor errors and normal operations, explains likely causes, rates severity, alerts the right person, and writes a daily review with every figure traced to source, so faults are fixed before they affect product or staff.

3 Alert Levels

Every alert is assigned one of four levels, which decides who is notified and how quickly, so urgent faults reach the right person and minor ones interrupt no one.

Level	Trigger	Notified	Response
Healthy	Reading inside the learned normal range	Daily review only	None
Watch	Recurring anomaly or slow drift	Daily review and plant lead	Next day
Attention	Food-safety room above 41°F for 30+ minutes, or many rooms at once	Plant manager and refrigeration tech (SMS, Slack)	1 hour
Critical	Ammonia alarm, receiver at 0% in production, compressor trip with no backup	On-call and plant manager, phone until acknowledged	Now

Thresholds and response times to be set with the client. Known sensor faults, planned maintenance and evaporators switched off on purpose never send an alert.

4 Requirements

The system must:

Data ingest and quality

- **R1** Read every nightly export from all three plants, in both file layouts, into one table: plant, date, report, point, hour, value and unit.
- **R2** Flag data-quality issues: dead or drifting sensors, duplicate points, odd units, and summaries that disagree between reports.
- **R3** Flag missing reports and report groups, and never fill the gap with old values.

Analysis and prediction

- **R4** Learn each point's normal range by hour of day from history, and compare every reading to that range and to its setpoint.
- **R5** Record every setpoint change with its time and its old and new values.
- **R6** Group simultaneous deviations into one event, with a likely cause and a severity.
- **R7** Use work orders, shift logs, schedules, weather, and dock and receiving records to explain events and suppress known causes.
- **R8** Forecast 24 to 72 hours ahead, flag gaps between actual and forecast, detect slow trends in recurring events, and explain misses.

Reporting, escalation and testing

- **R9** Write the daily review in the client's five-part format, with every figure linked to the reading it came from.
- **R10** Send alerts by level, track acknowledgement, and escalate any alert that is not acknowledged.
- **R11** Deliver the daily review by email and on an operations dashboard, and alerts by SMS, Slack or Teams.
- **R12** Generate months of test history with labeled, seeded faults, and score detection against them.

5 Success Criteria

What we check	Target (to confirm with the client)	How we check
Reads every file	All nightly reports from all 3 plants load with no manual cleanup (100%)	Parser log
Every number has a source	Every number links back to the raw data; mistakes in human reviews are flagged (100%)	The 6 sample reviews
Finds problems on its own	Catches sudden faults (like June 9), slow problems (like the 02:45 freezer, by day two) and 80%+ of test faults	Replay past data
No false alarms	No alerts from broken sensors, maintenance or planned shutdowns; max 2 false alerts per plant per week	Replay past data
Good forecasts	At least 15% more accurate than copying yesterday's numbers; explains big misses	Held-back test days
Runs on Contextual	Uses Object Types, Flows, Agents and a Site, start to finish	Final demo

6 Scope

In scope: all requirements, built on the Contextual platform. Models start with a rolling median and spread per hour and grow only where needed. Out of scope: the offline field app, and any automatic control of equipment or setpoints. The system advises people; it does not operate the plant.

Data: nightly controller exports (June samples and April to September history), the six sample reviews, and operations records: asset register, work orders, shift logs, alarm history, field rounds, ammonia detectors, dock doors, receiving, production schedule, weather and energy use.

7 Delivery Plan

Week	Phase	Technical work	Outcome
1	Ingest and inventory	Parsers for both layouts; point inventory; sample review checked	Nightly files readable without cleanup
2	Data quality and baselines	Quality flags; hourly normal ranges; seeded-fault test history	Sensor faults separated from real data
3	Detection and context	Event grouping; cause and severity; operations records linked	Real faults separated from normal activity
4	Reporting and alerts	Daily review generator; alert levels; simple forecast	Automatic reviews; right person alerted
5	Validation and handover	Replay history against seeded faults and six reviews; final demo	Success criteria met